

# Practical Test Assessment for Automation QA Engineer

**Role:** Semi-Senior QA Automation Engineer

**Duration:** 2 - 4 Hours (Approximate)

**Tools:** Playwright, TypeScript

**Application:** Product Manager App (amazonaws)

## Objective General:

The goal of this assessment is to evaluate your proficiency in **Playwright** and **TypeScript**, your ability to structure automation code using the **Page Object Model (POM)**, and your approach to testing web applications. We are looking for clean, maintainable, and reliable code.

## Objective Specifics:

This assessment evaluates your skills in automating user registration and login functionalities using Playwright and Typescript. You will demonstrate your ability to:

- Set up a Playwright project with Typescript.
- Automate user registration on a demo website.
- Registering new credentials taken from environment variables
- Automate login using previously registered credentials.
- Implement field validations on the login page.
- Utilize environment variables for secure credential handling.

## Tools and Technologies:

- Playwright
- Typescript
- Code Editor (Your Preference)
- NodeJS

## Preparation:

- An updated version of NodeJS should already be installed on the equipment where the assessment will be executed.
- The editor of the preference should already be pre-installed and ready.
- The candidate will have access to an online repository, where an account has to be registered and be able to create and manipulate a repository there.

## Tasks:

### Part I: Playwright Installation and Project Setup

01. On your local machine, create a new folder with the name “PW\_Assessment\_YYMMDD”

where:

YY = Last two digits of the current year.

MM = Current month from 01 to 12.

DD = Current day from 01 to 31.

for a script project.

Please consider using the date when the assessment was sent.

02. In the folder created in the previous step, install Playwright.

03. Set up an “.env” file to store base URLs and sensitive data (e.g., test credentials).

### Part II: UI Automation Scenarios

03. Write a Playwright test script in Typescript to automate user registration on the website:

<https://softwaremind-assessments.s3.us-east-1.amazonaws.com/login.html>

Implement the following scenarios. **Use of the Page Object Model (POM) is mandatory.**

*Scenario A: User Registration (Happy Path)*

1. Navigate to the **Login Page**.
2. Click the "Register here" link.
3. Register a user with a **randomized** username and email (to allow repeated execution).
  - *Tip: You can use a helper function to generate random strings.*
4. **Validate:** Verify that after registration, the user is redirected back to the Login Page.

## Part III: Login Automation and Field Validation

### Scenario B: Authentication (Negative & Positive)

1. **Negative Test:** Attempt to log in with an invalid username/password.
  - **Validate:** The error message "Invalid username or password" is displayed.
2. **Positive Test:** Log in with the user created in **Scenario A** (or use the demo credentials `admin` / `admin123` if you wish to separate dependencies).
  - **Validate:** Successful redirection to the **Dashboard** (`index.html`).

## Part IV: CRUD Flow

### Scenario C: Product Management (CRUD Flow)

*Pre-condition: User is logged in.*

1. **Create Product:**
  - Fill out the "Add New Product" form.
  - **Name:** "Automation Laptop [RandomID]"
  - **Price:** "1500"
  - **Category:** "Electronics"
  - **In Stock:** Checked (True)
  - Click "Save Product".
  - **Validate:** The new product appears in the Product List table.
2. **Filter:**
  - Use the "Filter by Category" dropdown to select "Electronics".
  - **Validate:** Your created product is still visible.
3. **Delete Product:**
  - Find the row containing your created product.
  - Click the **Delete** button for that specific row.
  - **Validate:** The product is removed from the table.

## Part V: Reporting & Code Quality

1. Ensure your `playwright.config.ts` is configured to generate an **HTML Report**.
2. Ensure screenshots or traces are captured on test failure.
3. Write a brief `README.md` explaining:
  - How to install dependencies.

- How to run the tests.
- Any trade-offs or decisions you made during implementation.

### Assessment Criteria:

- Functionality and correctness of test scripts.
- Code quality, structure, and maintainability.
- Effective use of TypeScript and Playwright libraries.
- Proper implementation of field validations.
- Secure handling of credentials with environment variables.
- **Code Structure:** clear separation of concerns using POM.
- **Selectors:** Use of resilient locators (e.g., `getByRole`, `getByPlaceholder`, `data-testid` where available) instead of brittle XPath.
- **Type Safety:** Proper use of TypeScript interfaces/types.
- **Reliability:** Handling of dynamic waits and asynchronous operations.
- **Documentation:** Clarity of your `README.md`.

### Additional Considerations:

- Be prepared to discuss your code and answer questions about the automation process.
- Feel free to use any additional libraries or frameworks that enhance your test automation approach.

### Success:

The successful completion of this assessment will showcase the candidate's capabilities in web test automation with Playwright and Typescript. It comprises the understanding of secure credential handling, field evaluation and manipulation, all of which will further demonstrate the skills of a qualified Automation QA Engineer.